

Practical 5

AIM: To Implementation of a knapsack problem using dynamic programming

1 ALGORITHM:

Algorithm:-
Input :- Number of items n, knapsack capacity w, weight array w and value array v
Output :- maximum possible value & the selected items
Step 1:- start
Step 2:- Read n, w, array w & array v
Step 3:- Initialize base case
(i) Set $dp[i][0] = 0$ for all i from 0 to n
(ii) Set $dp[0][j] = 0$ for all j from 0 to w
Step 4:- Repeat while $i < n$ (starting from $i = 1$)
(i) Set $j = 1$
(ii) Repeat while $j \leq w$
* if $w[i] \leq j$, set $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w] + v[i])$
* else set $dp[i][j] = dp[i-1][j]$
* increment j
Step 5:- Display dynamic programming table dp & max value $dp[n][w]$
Step 6:- Find selected items
i. set $j = w$
ii Repeat while $i \geq 1$ (starting from $i = n$)
* if $dp[i][j] \neq dp[i-1][j]$, then display "item i taken"
* else display "item i not taken"
* decrement j
Step 7:- Stop

2 PROGRAM:

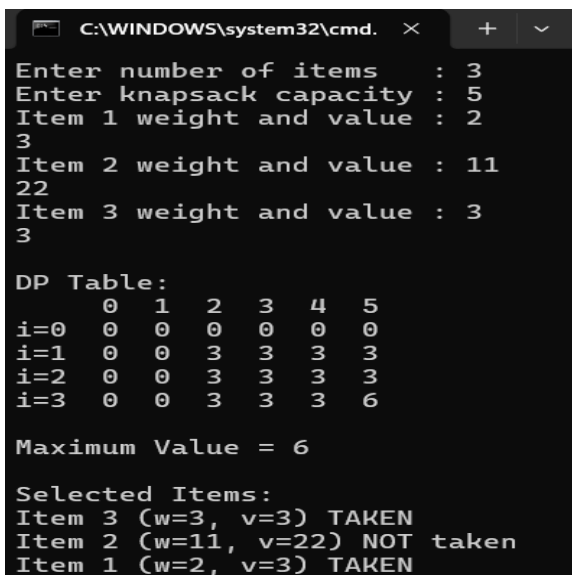
```
#include <iostream>
#include <vector>
using namespace std;
class Knapsack {
private:
    vector<int> w; // weights
    vector<int> v;
    int n, W;
    vector<vector<int>> dp;
public:
    void getInput() {
        cout << "Enter number of items : ";
        cin >> n;
        cout << "Enter knapsack capacity : ";
        cin >> W;
        w.resize(n + 1); v.resize(n + 1);
        dp.assign(n + 1, vector<int>(W + 1, 0));
        for (int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
```

```

    cin >> w[i] >> v[i];}
void solve() {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= W; j++) {
            if (w[i] <= j) {
                dp[i][j] = max(v[i] + dp[i - 1][j - w[i]], dp[i - 1][j]);
            } else {dp[i][j] = dp[i - 1][j];}}
    }
void printTable() {
    cout << "\nDP Table:\n ";
    for (int j = 0; j <= W; j++){cout << j << " ";}
    cout << endl;
    for (int i = 0; i <= n; i++) {
        cout << "i=" << i << " ";
        for (int j = 0; j <= W; j++)
            cout << dp[i][j] << " ";
        cout << endl;}}
void findItems() {
    cout << "\nSelected Items:\n";int j = W;
    for (int i = n; i >= 1; i--) {
        if (dp[i][j] != dp[i - 1][j]) {
            cout << "Item " << i << " (w=" << w[i] << ", v=" << v[i] << ")TAKEN\n";
            j -= w[i];} else {
    cout << "Item " << i << " (w=" << w[i] << ", v=" << v[i] << ")NOTtaken\n";}}}
void showResult() {
    cout << "\nMaximum Value = " << dp[n][W] << endl;}};
int main() {
    Knapsack k;
    k.getInput();k.solve();k.printTable();k.showResult();k.findItems();
    return 0;}

```

3 OUTPUT:



```

C:\WINDOWS\system32\cmd.  x  +  v
Enter number of items : 3
Enter knapsack capacity : 5
Item 1 weight and value : 2
3
Item 2 weight and value : 11
22
Item 3 weight and value : 3
3
DP Table:
   0  1  2  3  4  5
i=0  0  0  0  0  0  0
i=1  0  0  3  3  3  3
i=2  0  0  3  3  3  3
i=3  0  0  3  3  3  6
Maximum Value = 6
Selected Items:
Item 3 (w=3, v=3) TAKEN
Item 2 (w=11, v=22) NOT taken
Item 1 (w=2, v=3) TAKEN

```

4 TIME COMPLEXITY:

Time complexity

```

for i = 1 to n
  for j = 0 to W
    if W[i] <= j
      K[i][j] = max[K[i-1][j], K[i-1][j-W[i]] + P[i]]
    else
      K[i][j] = K[i-1][j]
  
```

so
 $T = n \times W \times O(1)$
 Ignoring constant
 $T = nW$

suppose:
 $n = 4 \text{ items}, W = 5 \text{ capacity}$
 no. of table calculation
 $n \times W \Rightarrow 4 \times 5 = 20$
 so, the algorithm

1. worst case:- every table cell is calculated
 $T(\text{worst}) = n \times W$
 if $n = 4, W = 5$
 $T(\text{worst}) = 4 \times 5 = 20$
 worst case = $O(nW)$

2. Average case:- even in an average case also the algorithm will go through all $n \times W$ table
 $T_{\text{average}} = n \times W$
 $n = 4, W = 5$
 $T = 4 \times 5 = 20$
 Average case = $O(nW)$

3. Best case $T(n \times W) = n \times W$
 * Best case = $O(n \cdot W)$
 * Average case = $O(n \cdot W)$
 * worst case = $O(n \cdot W)$

5 Conclusion:

The 0/1 Knapsack algorithm finds the combination of items having maximum profit without exceeding the given knapsack capacity